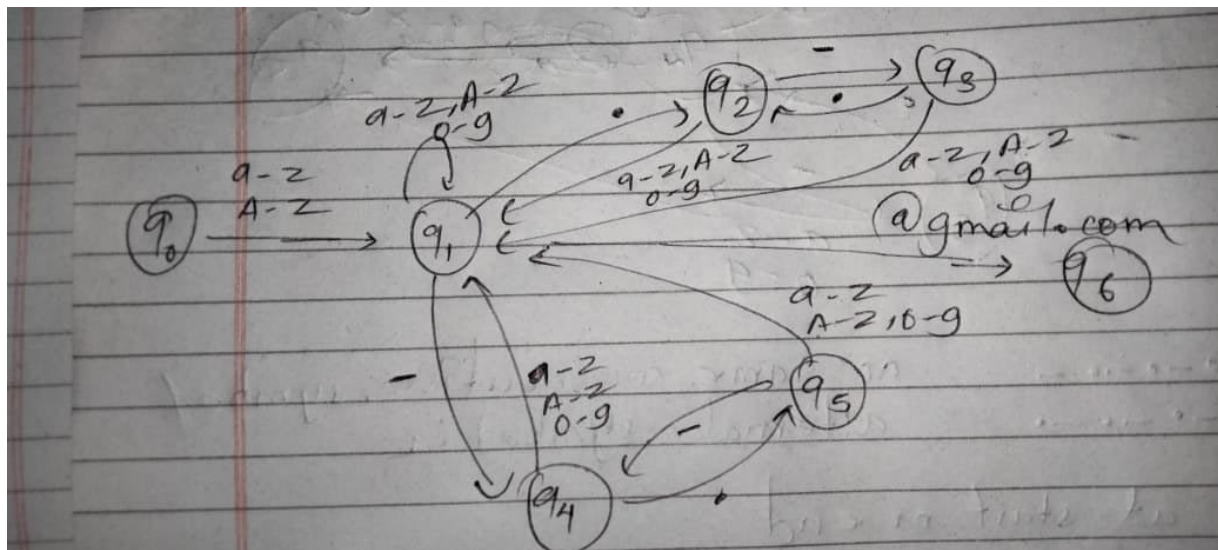
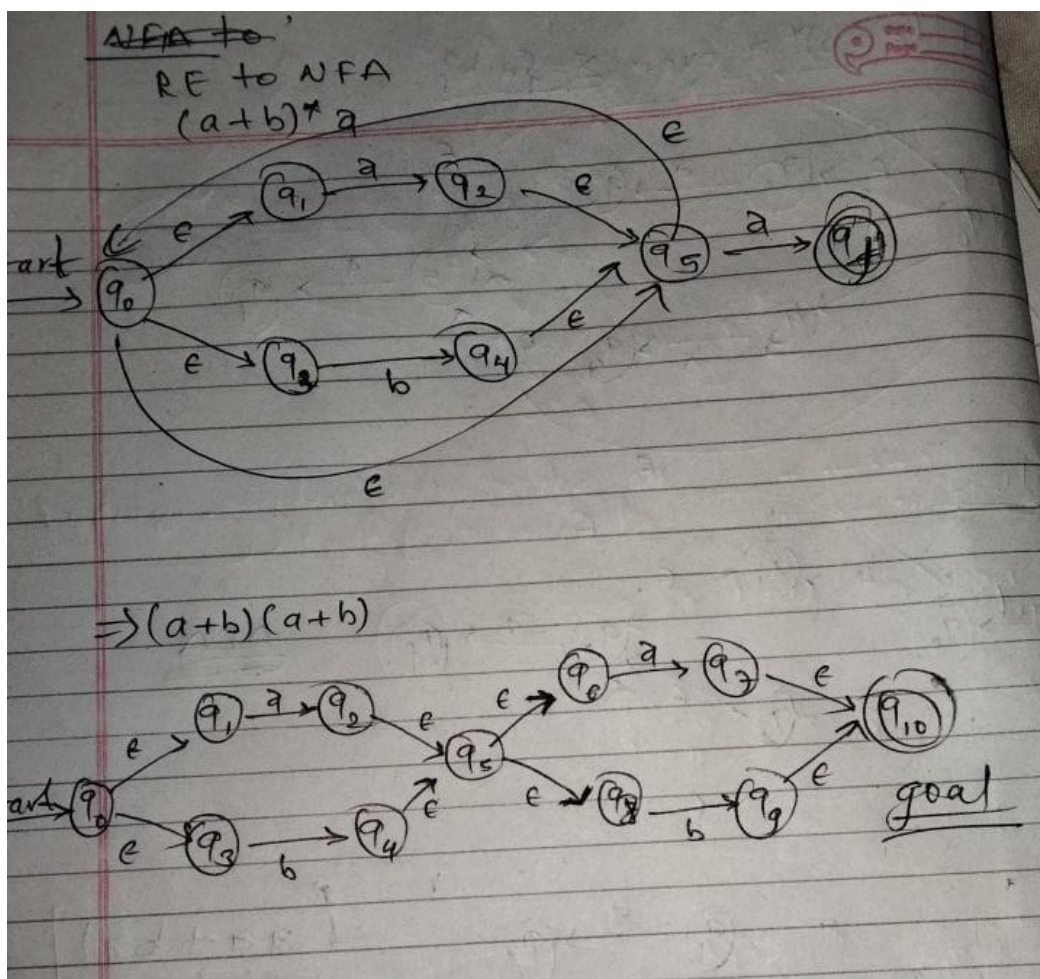


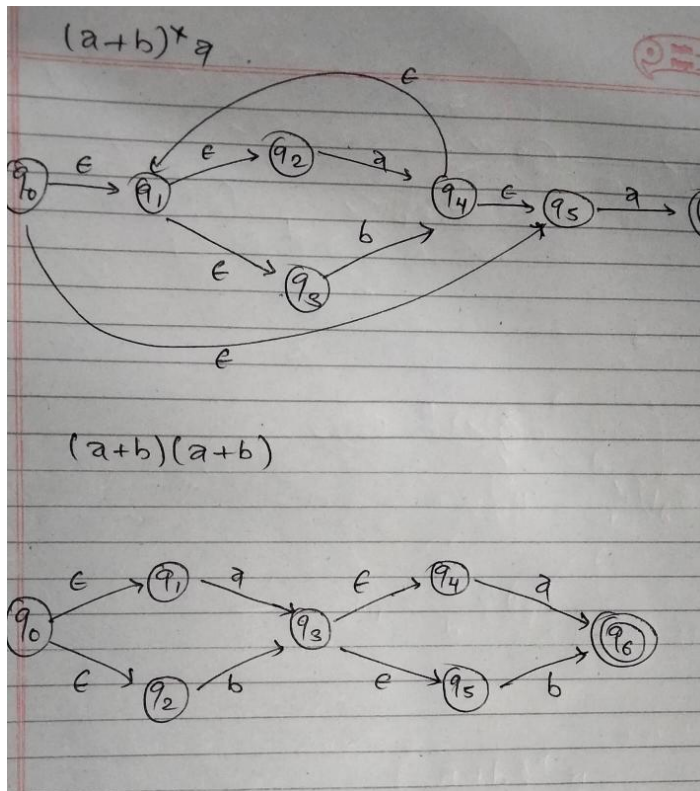
DFA accepting valid gmail address



NFA accepting RE $(a + b)^*a$ and $(a+b)(a+b)$ over $\Sigma = \{a,b\}$



More simpler



First NFA: $(a+b)^*a$ ✓

```
transitions = {
    0: {'': {1, 5}}, # Start: ε to (a+b)* entry OR skip directly to final 'a'
    1: {'': {2, 3}}, # Split for (a|b)
    2: {'a': {4}},   # a branch
    3: {'b': {4}},   # b branch (comment was slightly off)
    4: {'': {1, 5}}, # Loop back to (a+b)* OR proceed to final 'a'
    5: {'a': {6}},   # Final mandatory 'a'
    6: {}            # Accept state
}
```

Second NFA: $(a+b)(a+b)$ ✓

```
transitions = {
    0: {'': {1, 2}}, # ε-split to first (a+b)
    1: {'a': {3}},   # first a branch
    2: {'b': {3}},   # first b branch
    3: {'': {4, 5}}, # ε-split to second (a+b)
    4: {'a': {6}},   # second a branch
    5: {'b': {6}},   # second b branch
    6: {}            # final accepting state
}
```

Solution:

1. Design a DFA

a. Accepting binary string that start with 0 and ends with 1.

```
# DFA accepting binary string that starts with '0' and ends with '1'

print("DFA accepting binary strings that start with '0' and end with '1'.")
print("Enter a binary string: ", end='')
input_string = input().strip() # removes leading/trailing whitespace

state = 0 # initial state
print('DFA path: Q0', end=' ')
for char in input_string:
    if char not in '01':
        print("Invalid input. Please enter a binary string.")
        exit()
    else:
        if state == 0:
            if char == '0':
                state = 1
            else:
                state = 3
        elif state == 1:
            if char == '0':
                state = 1
            else:
                state = 2
        elif state == 2:
            if char == '0':
                state = 1
            else:
                state = 2
        elif state == 3:
            state = 3
        print(f"Q{state}", end=' ')

if state == 2:
    print("\nAccepted")
else:
    print("\nNot Accepted")
```

```
DFA accepting binary strings that start with '0' and end with '1'.
Enter a binary string: 00101
DFA path: Q0 Q1 Q1 Q2 Q1 Q2
Accepted
```

b. Accepting the valid variable names in C program

```
import re

def is_valid_c_variable_name(input_string):
    reserved_keywords = {'int', 'float', 'char', 'double', 'long', 'short', 'void',
                          'if', 'else', 'while', 'for', 'return'}
    if input_string in reserved_keywords:
        print(f"{input_string} is a reserved keyword.")
        return False

    if re.fullmatch(r'[a-zA-Z0-9_]+', input_string) is None:
        print("Symbols are invalid.")
        return False

    state = 0 # initial state
    print('DFA path: Q0', end=' ')
    for char in input_string:
        if state == 0:
            if re.fullmatch(r'[a-zA-Z]', char):
                state = 1
            elif char == '_':
                state = 2
            else:
                return False
        elif state == 1:
            if re.fullmatch(r'[a-zA-Z0-9_]', char):
                state = 1
            else:
                return False
        elif state == 2:
            if re.fullmatch(r'[a-zA-Z]', char):
                state = 1
            elif char == '_':
                state = 2
            else:
                return False
        print(f"Q{state}", end=' ')
    if state == 1:
        return True
    print("DFA accepting valid C variable names.")
    print("Enter a input string: ", end='')
    input_string = input().strip() # removes leading/trailing whitespace

    if is_valid_c_variable_name(input_string):
        print("\nString Accepted")
    else:
        print("\nNot a valid C variable name.")
```

```
DFA accepting valid C variable names.
Enter a input string:  _get_ans
DFA path: Q0 Q2 Q1 Q1 Q1 Q1 Q1 Q1 Q1
String Accepted
```

c. Accepting the valid gmail ID

```
import re

def is_valid_gmail(input_string):
    gmail = input_string.split('@')
    username = gmail[0]
    domain = '@'.join(gmail[1:]) # in case there are multiple '@' symbols

    print(username)
    print(domain)
    if not domain == 'gmail.com':
        print('Not a valid gmail address.')
        return False
    else:
        if not re.fullmatch(r'[a-zA-Z0-9_]+',
                           username): # username must only contain
            alphanumeric, dot(.) and underscore(-) character
            print("Not a valid username.")
            return False
        else:
            state = 0
            print('DFA path: Q0', end=' ')
            for char in username:
                if state == 0:
                    if re.match(r'[a-zA-Z]', char):
                        state = 1
                    else:
                        return False
                elif state == 1:
                    if char == '.':
                        state = 2
                    elif char == '_':
                        state = 4
                    elif re.match(r'[a-zA-Z0-9]', char):
                        pass # as it is (state = 1)
                    else:
                        return False
                elif state == 2:
                    if char == '_':
                        state = 3
                    elif re.match(r'[a-zA-Z0-9]', char):
                        state = 1
                    else:
                        return False
                elif state == 3:
                    if char == '.':
                        state = 2
                    elif re.match(r'[a-zA-Z0-9]', char):
                        state = 1
                    else:
                        return False
                elif state == 4:
                    if char == '.':
```

```

        state = 5
    elif re.match(r'[a-zA-Z0-9]', char):
        state = 1
    else:
        return False
    elif state == 5:
        if char == '_':
            state = 4
        elif re.match(r'[a-zA-Z0-9]', char):
            state = 1
        else:
            return False
    print(f"Q{state}", end=' ')

    return True    # will be at state 1 if not returned false on any
other state.

print('DFA accepting valid Gmail Id')
print("Enter a input string: ", end='')
input_string = input().strip() # removes leading/trailing whitespace

if is_valid_gmail(input_string):
    print("\nValid Gmail Address. (String Accepted)")

```

```

DFA accepting valid Gmail Id
Enter a input string:  git_hub.user@gmail.com
git_hub.user
gmail.com
DFA path: Q0 Q1 Q1 Q1 Q4 Q1 Q1 Q1 Q2 Q1 Q1 Q1 Q1
Valid Gmail Address. (String Accepted)

```

d. Accepting the prepaid NTC number

```
import re

def is_valid_prepaid_phone_no(input_string):
    if len(input_string) != 10 or not re.fullmatch(r'[0-9]+', input_string):
        print('Invalid Prepaid No.')
        return False
    elif input_string[:2] != '98':
        print('Not even a Nepali phone no.')
        return False
    else:
        state = 0
        print('DFA path: Q0', end=' ')
        for char in input_string[2:]:
            if state == 0:
                if char != '5' and re.match(r'[0-9]', char):
                    state = 1
                else:
                    return False
            elif state < 8 :
                if re.match(r'[0-9]', char):
                    state = state + 1
                else:
                    return False
            else:
                return False
        print(f"Q{state}", end=' ')

        return True    # will be at state 1 if not returned false on any
other state.

print('DFA accepting valid Prepaid Phone no.')
print("Enter a input string: ", end='')
input_string = input().strip() # removes leading/trailing whitespace

# input_string = '9815000098'

if is_valid_prepaid_phone_no(input_string):
    print("\nValid Prepaid no. (String Accepted)")
else:
    print("\nInvalid Prepaid no. (String Not Accepted)")
```

```
DFA accepting valid Prepaid Phone no.
Enter a input string: 9810213487
DFA path: Q0 Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8
Valid Prepaid no. (String Accepted)
```

2. NFA to simulate the following machine

- Accepting RE $(a + b)^*a$ over $\Sigma = \{a,b\}$
- Accepting RE $(a+b)(a+b)$ over $\Sigma = \{a,b\}$

```
# transition states for the NFA accepting the regular expression (a+b)*a
transitions_1 = {
    0: {'': {1, 5}},
    1: {'': {2, 3}},
    2: {'a': {4}},
    3: {'b': {4}},
    4: {'': {1, 5}},
    5: {'a': {6}},
    6: {}
}

# transition states for the NFA accepting the regular expression (a+b)(a+b)
transitions_2 = {
    0: {'': {1, 2}},          # ε-split to first (a+b)
    1: {'a': {3}},            # first a branch
    2: {'b': {3}},            # first b branch
    3: {'': {4, 5}},          # ε-split to second (a+b)
    4: {'a': {6}},            # second a branch
    5: {'b': {6}},            # second b branch
    6: {}                     # final accepting state
}

start_state = 0
accept_states = {6}         # same for both NFAs

def epsilon_closure(states, transitions):
    closure = set(states)
    stack = list(states)
    while stack:
        state = stack.pop()
        if state in transitions and '' in transitions[state]:
            for next_state in transitions[state]['']:
                if next_state not in closure:
                    closure.add(next_state)
                    stack.append(next_state)
    return closure

def nfa_accepts(input_string, transitions):
    current_states = epsilon_closure({start_state}, transitions)

    for symbol in input_string:
        next_states = set()
        for state in current_states:
            if state in transitions and symbol in transitions[state]:
                next_states.update(transitions[state][symbol])
        current_states = epsilon_closure(next_states, transitions)

    return bool(current_states & accept_states)
```



```

# Test
test_strings = ['', 'a', 'b', 'aa', 'ab', 'ba', 'cd']

print("Testing NFA for (a+b)*a:")
for str in test_strings:
    result = "ACCEPT" if nfa_accepts(str, transitions_1) else "REJECT"
    print(f"'{str}' -> {result}")

print("\n----- Testing NFA for (a+b)(a+b): -----")
for str in test_strings:
    result = "ACCEPT" if nfa_accepts(str, transitions_2) else "REJECT"
    print(f"'{str}' -> {result}")

```

```

Testing NFA for (a+b)*a:
'' -> REJECT
'a' -> ACCEPT
'b' -> REJECT
'aa' -> ACCEPT
'ab' -> REJECT
'ba' -> ACCEPT
'cd' -> REJECT

----- Testing NFA for (a+b)(a+b): -----
'' -> REJECT
'a' -> REJECT
'b' -> REJECT
'aa' -> ACCEPT
'ab' -> ACCEPT
'ba' -> ACCEPT
'cd' -> REJECT

```

3. Left Recursive Grammar

```
# Program to check and eliminate left recursion from a given grammar

print("Enter a left recursive grammar...(e.g. S = Sab | ab | a | b) ")
grammar = input().strip()

# grammar = 'S = Sab | ab | a | b'
# grammar = 'B = BaaC | Bd | ef | g'
# grammar = 'A = A0 | A1 | 0'

l_s, r_s = grammar.split('=')
l_s = l_s.strip()
r_s = r_s.strip()

productions = [prod.strip() for prod in r_s.split('|')]    # splitting
productions

for prod in productions:
    if prod[0] == l_s:
        print(f"{grammar} is left recursive.")
        break
    else:
        # runs only if the loop completes normally (i.e., no break
        occurred). => not left recursive
        print(f"{grammar} is not left recursive.")
        exit() # to stop further execution if not left recursive

# Eliminating left recursion
print("Eliminating left recursion...")

new_prod = []
prod_dash = []

for prod in productions:
    if prod[0] == l_s:
        prod_dash.append(prod[1:] + l_s + "'")    # A -> αA'
    else:
        new_prod.append(prod + l_s + "'")    # A -> βA'

print('\n***** Production after eliminating left recursion: *****')
print(f"{l_s} = {' | '.join(new_prod)}")
print(f"{l_s}' = {' | '.join(prod_dash) + ' | ε\n'}")
# adding ε production for A'
```

Enter a left recursive grammar...(e.g. $S = Sab \mid ab \mid a \mid b$)

$S = Sab \mid ab \mid a \mid b$

$S = Sab \mid ab \mid a \mid b$ is left recursive.

Eliminating left recursion...

***** Production after eliminating left recursion: *****

$S = abS' \mid aS' \mid bS'$

$S' = abS' \mid \epsilon$

Enter a left recursive grammar...(e.g. $S = Sab \mid ab \mid a \mid b$)

$A = A0 \mid A1 \mid 0$

$A = A0 \mid A1 \mid 0$ is left recursive.

Eliminating left recursion...

***** Production after eliminating left recursion: *****

$A = 0A'$

$A' = 0A' \mid 1A' \mid \epsilon$

4. Symbol Table

```
# Write a program to create a symbol table for the variables (for data types
only)
# Example: - Input (int a = 2, float b = 3.5), output will be as
# Variable Name Type Value Address
# a integer 2 100
# b real number 3.5 200

# print("Enter variable declarations (e.g. int a = 2, float b = 3.5): ")
# declarations = input().strip()

declarations = 'int a = 2, float b = 3.5, char c = \'x\', double d = 4.56,
long e = 1234567890, short f = 12, byte g = 255, bool h = true, string i =
"hello"'
declarations_list = [decl.strip() for decl in declarations.split(',')]
print("\nVariable Declarations:")
for decl in declarations_list:
    print(decl)

symbol_table = []
address_counter = 100
for decl in declarations_list:
    parts = decl.split()
    var_type = parts[0]
    var_name = parts[1]
    var_value = parts[3]
    symbol_table.append((var_name, var_type, var_value, address_counter))
    address_counter += 100

print("\nSymbol Table:")
print(f"{'Variable Name':<15} {'Type':<15} {'Value':<10} {'Address':<10}")
for entry in symbol_table:
    print(f"{entry[0]:<15} {entry[1]:<15} {entry[2]:<10} {entry[3]:<10}")
```

Variable Declarations:

int a = 2

float b = 3.5

char c = 'x'

Symbol Table:

Variable Name	Type	Value	Address
a	int	2	100
b	float	3.5	200
c	char	'x'	300

Variable Declarations:

```
int a = 2
float b = 3.5
char c = 'x'
double d = 4.56
long e = 1234567890
short f = 12
byte g = 255
bool h = true
string i = "hello"
```

Symbol Table:

Variable Name	Type	Value	Address
a	int	2	100
b	float	3.5	200
c	char	'x'	300
d	double	4.56	400
e	long	1234567890	500
f	short	12	600
g	byte	255	700
h	bool	true	800
i	string	"hello"	900

5. Context Free Grammar

```
grammar = 'S = Sab | ab | a | b'
print("Given Grammar: ", grammar)

V = set() # set of variables (set will avoid duplicates)
T = set() # set of terminals
productions_dict = dict() # dictionary to hold productions
productions_set = set() # set to hold all productions
starting_symbol = ''

# Parsing the grammar
l_s, r_s = grammar.split('=')
l_s = l_s.strip()
r_s = r_s.strip()

if l_s.isupper():
    starting_symbol = l_s
    V.add(l_s) # adding variable to V
    productions = [prod.strip() for prod in r_s.split('|')] # splitting productions
    for prod in productions:
        for char in prod:
            if char.isupper():
                V.add(char)
            else:
                T.add(char) # adding terminal to T
        productions_set.add(l_s + '->' + prod)
else:
    print("Invalid Grammar: Left side must be a variable (uppercase letter).")
    exit()

print("Starting Symbol: ", starting_symbol)
print("Variables (V): ", V)
print("Terminals (T): ", T)
print("Productions: ", productions_set)
```

```
Given Grammar: S = Sab | ab | a | b
Starting Symbol: S
Variables (V): {'S'}
Terminals (T): {'a', 'b'}
Productions: {'S->Sab', 'S->a', 'S->ab', 'S->b'}
```

6. Loop Optimization

a. Loop Motion

```
# Code Motion (Moves computations that do not change within the loop outside
of the loop.)
import time

n = 30000000    # 30 lakh
a = b = d = e = f = g = h = 0
start = time.time() # Record start time
for i in range(n):
    a = b + i
    c = d + e
    f = g + h

end = time.time()    # Record end time

print("***** Code Motion *****")
print("Execution Time (Before Code Motion):", end - start, "seconds")

a = b = d = e = f = g = h = 0
start = time.time() # Record start time

c = d + e    # Moves computations that do not change within the loop outside
of the loop.
f = g + h
for i in range(n):
    a = b + i
end = time.time()    # Record end time

print("Execution Time (After Code Motion):", end - start, "seconds")
```

```
***** Code Motion *****
```

```
Execution Time (Before Code Motion): 8.01967191696167 seconds
```

```
Execution Time (After Code Motion): 3.494628429412842 seconds
```

b. Loop Jamming

Loop Jamming (Loop Fusion) -> is an optimization technique where two or more loops that iterate over the same range are combined into a single loop. This can improve performance by reducing loop overhead and enhancing cache locality.

```
import time

n = 10000000    # one crore
squares = []
doubles = []

# Separate loops
start = time.time()
for x in range(n):
    squares.append(x ** 2)
for x in range(n):
    doubles.append(x * 2)

end = time.time()

print("***** Code Jamming *****")
print("Execution Time (Before Loop Jamming):", end - start, "seconds")

# Fused loop (loop jamming)
squares = []
doubles = []
start = time.time()
for x in range(n):    # Combines two or more loops that iterate over the
    # same range into a single loop to reduce overhead.
    squares.append(x ** 2)
    doubles.append(x * 2)

end = time.time()
print("Execution Time (After Loop Jamming):", end - start, "seconds")
```

```
***** Code Jamming *****
Execution Time (Before Loop Jamming): 4.029698133468628 seconds
Execution Time (After Loop Jamming): 2.8633618354797363 seconds
```